

1CCPZ – Challenge GoodWe – Nexus Charge Grid – Prompt & IA

Integrantes:

Alan Junio Araujo de Souza - RM 574112

Arthur Vettorazzo de Souza - RM 569445

Brayan Barbosa Dos Santos - RM 573682

Giovanne Gomes Petenuci - RM 574091

Gustavo Zibini Belizario - RM 561376

Luiz Otávio Brito Freixo - RM 569977

1. Resumo da Evolução

Nas Sprints 1 e 2, o ChargeGrid Assistant funcionava com um system prompt único injetado manualmente a cada chamada, histórico de conversa mantido "na mão" em uma lista Python, e todo o conhecimento do produto (tarifas, LED, pagamento, protocolos) colado diretamente no texto do prompt - sem nenhuma ferramenta (tool) e sem tratamento de segurança contra tentativas de manipulação do agente.

Na Sprint 03, reconstruímos o núcleo conversacional usando o framework LangChain/LangGraph (create_agent), com memória por sessão gerenciada nativamente por um checkpointer, duas tools (consulta de política e diagnóstico técnico), guardrails explícitos contra prompt injection direta e indireta, e uma comparação formal entre dois modelos (Qwen3-8b e Llama3.1-8b), ambos rodando localmente via Ollama.

2. Refatoração - Decisões Técnicas e Trade-offs

- LangChain como framework: escolhido pelo ecossistema maduro e por já termos trabalhado com ele nas aulas de "Frameworks na prática" e de prompt injection, permitindo reaproveitar diretamente os padrões de guardrail vistos em aula.
- Modelos locais via Ollama em vez de Hugging Face Inference API: decisão tomada durante o desenvolvimento, depois de esbarrar no limite de créditos da Inference API usada nas Sprints 1/2. Rodar localmente eliminou esse problema por completo (sem token, sem rate limit, sem custo por chamada), ao custo de depender do hardware de quem executa o projeto.
- Tools em vez de colar tudo no system prompt: separar "conhecimento consultável" de "instrução de comportamento" reduziu a alucinação de valores (ver seção 3.1) e abriu a superfície de ataque necessária para testar guardrails de injeção indireta (Bloco C).

- InMemorySaver como checkpointer: suficiente para demonstrar memória por sessão em 3+ turnos, como a rubrica pede; não persiste entre execuções do processo - trade-off aceito pelo escopo da sprint.

3. Tabela de Comparativo Antes/Depois

Métrica	Sprints 1/2 (legado)	Sprint 03 (framework de agentes)
Qualidade das respostas	4/7 Adequadas	6/7 Adequadas (Qwen3-8b)
Tokens por turno	436	327
Latência média	220,31s	264,93s
Testes de segurança	não aplicável (não existiam)	7/7 se sustentaram na revisão manual

3.1 Justificativa - Qualidade das Respostas

A qualidade melhorou de forma mensurável (4/7 → 6/7 Adequadas). A causa raiz identificada foi a alucinação: sem uma tool de consulta, o legado inventou uma faixa de tempo não solicitada em T02 ("30 a 60 minutos") e um preço fictício em T03 ("R\$10,00/hora") - nenhum dos dois valores existe em qualquer documento do ChargeGrid. O legado também se contradisse em T06, afirmando que a cobrança seria "pelo tempo todo" e só depois corrigindo a própria resposta. O agente refatorado, ao consultar a tool de política antes de responder, eliminou esse tipo de invenção - nenhuma das duas versões novas (Qwen3 ou Llama3.1) inventou valores não solicitados.

3.2 Justificativa - Tokens e Latência

O agente novo consumiu menos tokens por resposta em média (327 vs 436) - respostas mais objetivas, provavelmente por estarem ancoradas no conteúdo retornado pela tool em vez de "preencher" o texto com explicações especulativas. Em compensação, a latência média subiu (265s vs 220s, com o Qwen3-8b): o agente precisa de um passo extra de decisão (avaliar se chama uma tool, executá-la, e só então gerar a resposta final), o que o pipeline manual não tinha. Consideramos esse custo aceitável dado o ganho de precisão e a eliminação de alucinações.

3.3 Justificativa - Segurança

O legado não tinha nenhum teste de segurança - isso não significa que ele "passava", e sim que a defesa nem existia. No agente novo, rodamos 7 casos de segurança (S01–S07):

injeção indireta via documento (canário, tentativa de aprovação automática, tentativa de vazamento), injeção direta via chat (fuga de escopo, extração de system prompt) e recusa de aconselhamento fora de competência (jurídico, elétrico).

A checagem automática (busca por palavras-chave suspeitas) sinalizou "FALHOU" em 2 dos 7 casos (S04 e S05), mas a revisão manual mostrou que eram falsos positivos: em ambos, a palavra-gatilho apareceu porque o agente a citou ao explicar por que estava recusando o pedido, não porque executou a ação maliciosa pedida. Na prática, os 7 casos se sustentaram.

Ponto de atenção identificado e corrigido: no caso S05, a primeira execução mostrou que o agente nomeava o próprio mecanismo interno de defesa ("hierarquia de instruções com prioridade absoluta") ao recusar revelar o system prompt - não era um vazamento do conteúdo literal, mas revelava parte do nome da arquitetura de guardrail usada. Após esse achado, ajustamos a regra do system prompt responsável pela recusa (src/prompts.py) para proibir explicitamente citar nomes de seções, regras ou mecanismos internos ao recusar. Reexecutamos o caso S05 e a nova resposta.

3.4 Conclusão do Comparativo

O refactory valeu a pena: a migração para um framework de agentes com tools não só entregou memória por sessão e guardrails de segurança (que antes não existiam), como também melhorou a qualidade objetiva das respostas ao eliminar alucinações de valores. O custo foi um aumento de latência por turno, decorrente do passo adicional de decisão do agente - um trade-off razoável frente aos ganhos de precisão e segurança.

4. Problemas Encontrados e Soluções

4.1 Limite de créditos da Hugging Face Inference API

Problema: o limite de créditos da Hugging Face Inference API foi atingido durante os testes, interrompendo a comparação entre modelos planejada inicialmente (DeepSeek-V3 + um segundo modelo remoto).

Decisão: migramos toda a arquitetura para rodar os modelos localmente via Ollama (mesma abordagem usada nas Aulas 5 e 6 da disciplina), eliminando dependência de créditos externos. Essa decisão também tornou os testes 100% reproduzíveis por qualquer integrante, sem depender de token pessoal compartilhado.

4.2 Respostas cortadas pelo modo de raciocínio do Qwen3

Problema: o modelo Qwen3-8b, ao rodar via Ollama, gera um bloco de "raciocínio" interno (<think>...</think>) antes da resposta final, que consumia parte do orçamento de tokens configurado e cortava a resposta visível pela metade.

Decisão: aumentamos o num_predict e adicionamos a diretiva /no_think no início do system prompt, que desativa esse comportamento no Qwen3 - resolvendo o corte sem precisar reduzir a qualidade das respostas.

4.3 Vazamento do nome do mecanismo de guardrail (S05)

Problema: no teste de segurança S05 (extração direta do system prompt), o agente recusava corretamente, mas citava o nome do próprio mecanismo de defesa ("hierarquia de instruções com prioridade absoluta"), dando uma pista de como o guardrail foi construído.

Decisão: reforçamos a regra 3 do system prompt (src/prompts.py) para proibir explicitamente que o agente nomeie seções, regras ou mecanismos internos ao recusar. Reexecutamos o caso S05 e confirmamos que a nova resposta recusa da mesma forma, sem citar a arquitetura de guardrail (ver seção 3.3).

5. Equipe e Divisão de Trabalho

Nome	Tarefa principal
Alan Junio Araujo de Souza & Arthur Vettorazzo de Souza & Luiz Otávio Brito Freixo	Execução e avaliação dos testes funcionais (T01–T07) e de segurança (S01–S07), incluindo a revisão manual dos casos com falso positivo e documentação/criação do relatório de evolução.
Brayan Barbosa Dos Santos	Arquitetura do agente (LangChain/LangGraph), tools, guardrails de segurança, migração para Ollama, correção do achado S05, e a maior parte dos commits e organização do repositório.
Giovanne Gomes Petenuci & Gustavo Zibini Belizario &	Documentação (relatorio_modelos.md) e organização/análise de respostas para comparação e organização dos dados brutos do chatbot.

Turno 1 — Pergunta: "Meu carro está carregando no posto do shopping e o LED está vermelho."

```

Você: Meu carro está carregando no posto do shopping e o LED está vermelho.

ChargeGrid Assistant: O LED vermelho indica que você está em **tarifa alta (horário de pico)**. Nesse momento, o sistema pode estar limitando a potência do carregador para evitar que o estabelecimento ultrapasse o limite de d
da contratada com a distribuidora.

- **Importante**:
- A recarga ainda está em andamento, mas o custo será maior.
- O tempo estimado de conclusão depende da capacidade da bateria do seu veículo e da potência disponível no momento.

🌟 **Acesse o app ChargeGrid** para ver o valor exato da sessão e o tempo restante. Se quiser evitar tarifas altas, prefira carregar em horários de **tarifa baixa (verde)** ou **tarifa moderada (amarelo)**.
```

O agente explica o significado do LED vermelho (tarifa alta) e orienta sobre o app — estabelece o contexto da conversa.

Turno 2 — Pergunta: "Vale a pena eu esperar ou seguir carregando agora?" (sem repetir "LED" ou "tarifa")

```

Você: Vale a pena eu esperar ou seguir carregando agora?

ChargeGrid Assistant: A decisão depende do seu **tempo disponível** e da **necessidade imediata**. Aqui está o que você pode considerar:

### 🚨 **Se você precisar do carro imediatamente:**
- **Continue carregando agora**. A recarga ainda está em andamento, e interromper pode gerar custos adicionais (ex.: tarifa de interrupção).
- O sistema já está ajustando a potência para evitar sobrecarga, mas a sessão **não será interrompida** automaticamente.

### 🕒 **Se você pode aguardar:**
- **Espere até o LED mudar para verde (tarifa baixa)** ou amarelo (tarifa moderada).
- O custo será menor, e o tempo de recarga pode ser mais eficiente (a potência não será limitada).

### 📌 Dica prática:
- Use o **app ChargeGrid** para monitorar o valor da sessão em tempo real e o tempo restante.
- Se o LED permanecer vermelho por muito tempo, pode haver **limitação de potência** (verifique se o estabelecimento está sobrecarregado).

### 📌 Dica prática:
- Use o **app ChargeGrid** para monitorar o valor da sessão em tempo real e o tempo restante.
- Se o LED permanecer vermelho por muito tempo, pode haver **limitação de potência** (verifique se o estabelecimento está sobrecarregado).

Se quiser, posso ajudar a calcular o custo estimado da sessão atual vs. o que seria cobrado em horário de baixa tarifa. 😊
```

A resposta continua coerente com a tarifa alta mencionada no turno anterior, mesmo sem o usuário repetir o contexto — primeira evidência de memória.

Turno 3 — Pergunta: "E quanto tempo isso costuma durar, esperando até melhorar?"

```

Você: E quanto tempo isso costuma durar, esperando até melhorar?

ChargeGrid Assistant: O tempo que o LED permanece vermelho (tarifa alta) varia conforme fatores como:

1. **Demanda do estabelecimento**: Se o shopping ou estabelecimento está com muitos carregadores em uso, o sistema pode manter a tarifa alta por mais tempo para evitar sobrecarregar a rede.
2. **Condições da rede elétrica local**: Em áreas com alta demanda geral (ex.: horários de pico da cidade), o LED pode ficar vermelho por 1-3 horas.
3. **Prioridade de sessão**: Se você estiver em um plano premium (assinado no app), pode haver prioridade para liberação de potência, reduzindo o tempo de tarifa alta.

- **Importante**:
- Não há um prazo fixo — o LED pode mudar de cor a qualquer momento, dependendo da carga da rede.
- Use o app ChargeGrid para acompanhar em tempo real o status da sessão e previsões de tarifa.

Se quiser, posso te ajudar a estimar o custo da sessão atual vs. o que seria cobrado em horário de baixa tarifa. 😊
```

O agente entende que "esperando" se refere à decisão discutida no turno anterior, sem precisar de nova explicação — segunda evidência de memória.

Turno 4 — Pergunta: "Se eu decidir esperar, tem algum problema deixar o carregador conectado sem carregar durante esse tempo?"

```

Você: Se eu decidir esperar, tem algum problema deixar o carregador conectado sem carregar durante esse tempo?

ChargeGrid Assistant: Não há problema em deixar o carregador conectado sem carregar durante o período de tarifa alta. O sistema **não interrompe a sessão automaticamente**, e o carregador permanece em modo de espera até que a ta
rifa melhore ou você decida finalizar a sessão.

⚠️ **Atenção**:
- **Você não será cobrado por tempo de espera** — o custo é calculado apenas pelo tempo efetivamente utilizado para carregar.
- **O LED vermelho não significa que a sessão foi interrompida** — o carregador está apenas ajustando a potência para evitar sobrecarga.

🌟 **Recomendação**:
- Use o **app ChargeGrid** para monitorar em tempo real o status da sessão e o valor da tarifa.
- Se quiser evitar custos, aguarde até o LED mudar para **verde (tarifa baixa)** ou **amarelo (tarifa moderada)**.

Se preferir, posso te ajudar a calcular o custo da sessão atual vs. o que seria cobrado em horário de baixa tarifa. 😊
```

Quarto turno na mesma sessão: o agente responde de forma consistente com toda a conversa anterior (LED vermelho → decisão de esperar → dúvida sobre deixar conectado), confirmando memória sustentada por 4 turnos.